
Search and Verify: Efficient Small Object Tracking in 4K Video via Motion-Gated Native Tiling

Department of Computer Science
University of Oxford

Abstract

Tracking small objects in high-resolution video remains a significant challenge due to the loss of spatial information during standard image resizing. In this paper, we address the task of tracking birds in 4K video sequences under realistic CPU-only compute constraints. We evaluate thirteen distinct strategies ranging from naive deep learning baselines to hybrid pipelines involving Global Motion Compensation (GMC), dynamic statistical thresholding, and temporal interpolation. Our findings demonstrate that while brute-force Slicing Aided Hyper Inference (SAHI) dramatically increases recall from 0.10% to 54.50%, it incurs a heavy computational penalty. We propose Strategy 8 (Motion-Guided ROIs) and Strategy 12 (Temporal Interpolation) as optimal solutions, achieving the highest F1-score (8.29%) and highest throughput (3.54 FPS), respectively.

1 Introduction

Tracking small targets, such as birds in 4K video, is a fundamental problem in computer vision with applications in conservation and wind farm safety [1]. The primary difficulty lies in the scale; a bird may occupy only a few pixels in a 3840×2160 frame [1]. Standard detectors like YOLO typically resize input to 640×640 , causing these tiny features to vanish. This assignment explores methods to preserve native resolution while maintaining computational efficiency on CPU-only hardware [2].

2 Methodology

Our methodology follows a “Search and Verify” paradigm. We conducted extensive experiments involving many permutations of strategies, confidence thresholds (0.01 to 0.20), and variations in sharding and merging. We define a singular true baseline for comparison, with all other methods treated as distinct experimental strategies.

2.1 Baseline: `baseline_base`

The true baseline for all experiments is the **baseline_base** pipeline. It performs standard YOLOv12n inference on 4K frames resized directly to 640×640 . This establishes the lower bound for performance using native deep learning techniques without specialized handling for small objects.

2.2 Strategy 2: GMC and Dynamic Thresholding (`strategy_2.py`)

This hybrid pipeline utilizes Global Motion Compensation (GMC). It detects Shi-Tomasi corners and tracks them between $Frame_{t-1}$ and $Frame_t$ using Lucas-Kanade Optical Flow. By canceling camera motion, we isolate moving objects using an adaptive threshold $T = \mu + k\sigma$.

2.3 Native Tiling (baseline_w_tiling)

To mitigate resolution loss, we implemented a custom tiling strategy. The 4K frame is manually divided into a grid (e.g., 4x3) with 20% overlap to ensure continuity across seams. Each tile is processed at native resolution, and detections are mapped back to global coordinates.

2.4 SAHI Tiling (baseline_base_sahi)

In this strategy, we utilize the Slicing Aided Hyper Inference (SAHI) library to handle the sharding of the 4K frame. This serves as a comparison point for our manual native tiling implementation, testing the efficiency of library-based patch management.

2.5 Native Tiling + NMS (baseline_w_tiling_and_nms)

This is an extension of the Native Tiling strategy that adds a Global Non-Maximum Suppression (NMS) pass after coordinate mapping. This step is critical for merging duplicate detections that occur in the overlap regions between tiles.

2.6 Strategy 8: Motion-Guided ROIs and Cascaded Gates

Strategy 8 employs GMC to identify moving blobs, generating Regions of Interest (ROIs) that are expanded by 2.0x for context. Inference is throttled to every 5 frames, with a custom Object Tracker maintaining state. A full frame scan is performed every 10 frames to recover from motion-mask failures.

2.7 Strategy 10: Motion-Gated Native Tiling

This strategy applies native 640x640 tiling but only processes tiles flagged with significant motion via GMC stabilization. This focuses compute on active regions of the 4K frame.

2.8 Strategy 10 SAHI: Motion-Gated SAHI Tiling

This variation of Strategy 10 utilizes the SAHI library to generate the tiles that are subsequently gated by the motion detection logic.

2.9 Strategy 11: Classifier before Detection

Strategy 11 adds a “Fail Fast” logic. Candidate ROIs from motion detection are passed to a lightweight YOLOv11n-cls classifier. If the classifier does not identify a bird, the heavy detector pass is skipped.

2.10 Strategy 12: Interpolation with GMC

Based on Strategy 8, this method adds **Linear Interpolation**. By running inference every 5 frames and skipping the middle 4, it mathematically predicts the intermediate locations of a bird if it is detected in both the current and previous keyframes. Once every 10 frames a full frame scan is performed to recover from motion-mask failures.

2.11 Strategy 12 SAHI: Interpolation with SAHI

This variation swaps the GMC-based ROI generation for SAHI tiling while maintaining the linear interpolation logic between skipped frames.

2.12 Strategy 13: Kitchen sink with Native tiling

Our most complex funnel, combining native tiling, GMC motion detection, and classification. It processes tiles individually; if motion is detected, it goes to the detector; if not, a classifier checks for stationary birds.

2.13 Strategy 13: Kitchen sink with Native tiling and interpolation

This version of the "Kitchen sink" adds the temporal linear interpolation to the Native tiling funnel to further boost throughput.

2.14 Strategy 13 SAHI: Kitchen sink with SAHI tiling

The full funnel implementation using the SAHI library for initial image slicing instead of manual native tiling.

2.15 Strategy 13 SAHI: Kitchen sink with SAHI tiling and interpolation

The most advanced variant, utilizing SAHI slicing, the multi-gate funnel (motion/classification), and linear interpolation for maximum theoretical recall and efficiency.

3 Experiments and Results

3.1 Experimental Setup

Experiments were conducted on the SMOT4SB dataset using a Google Cloud VM (c4-standard-48). We evaluated thresholds of 0.01, 0.05, 0.1, and 0.2 across all strategies.

3.2 Performance Analysis

Table 1 summarizes the most significant strategies compared to the baseline.

Table 1: Performance Benchmarks on SMOT4SB Dataset

Strategy	Avg FPS	Precision	Recall	F1-Score
Baseline Base (0.01)	0.69	3.41%	0.10%	0.19%
Baseline Base + SAHI	0.44	2.39%	54.50%	4.58%
Strategy 2 + SAHI	0.03	2.15%	62.30%	4.15%
Strategy 8 + SAHI	1.82	4.65%	38.66%	8.29%
Strategy 12 (0.01)	3.54	7.85%	0.87%	1.56%
Strategy 12 + SAHI	2.25	2.53%	43.89%	4.78%

3.3 Discussion

The Slicing Paradox: SAHI remains the most effective method for increasing recall (+54,400% over baseline), but its 0.44 FPS throughput is unsuitable for real-time use. **Temporal Efficiency:** Strategy 12 achieved **3.54 FPS** using linear interpolation.

4 Conclusion

Our results indicate that **Strategy 8 with SAHI** provides the best balance of detection quality (8.29% F1), while **Strategy 12** offers the highest efficiency for CPU-based tracking.

References

- [1] MVA2025 Organizing Committee. Small multi-object tracking for spotting birds (smot4sb) challenge. <https://mva-org.jp/mva2025/challenge>, 2025. Official dataset source for the assignment [cite: 90, 91].
- [2] Ronald Clark. Computer vision (vis) assignment: Small object tracking. 2025. Assignment brief and learning objectives [cite: 12, 28].